

Der Tomaten-Bewässerungswächter als eigenständiges Gerät

Band 2: Vom PC-Programm zum Dauerläufer mit Anzeige und Klemmenebene

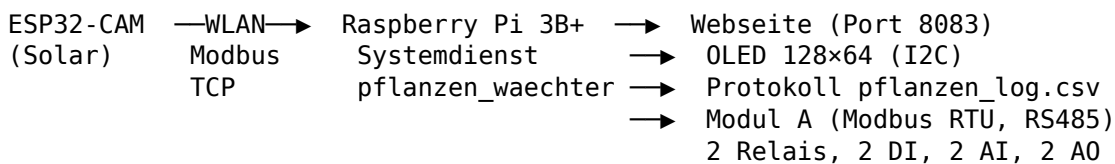
Ralph Wystup, August 2026

Einordnung

Der erste Band beschreibt das Verfahren: eine ESP32-CAM beobachtet eine Tomatenpflanze, überträgt ihre Bilder blockweise und CRC-gesichert über Modbus TCP, und ein Python-Programm trennt die Pflanze über den Excess-Green-Index vom Hintergrund, um aus Grünfläche und Schwerpunktlage den Wasserbedarf abzuleiten. Das alles gilt unverändert und wird hier nicht wiederholt.

Dieser Band behandelt, was danach kam: aus dem Programm, das auf einem eingeschalteten Windows-Rechner lief, wurde ein Gerät. Es besteht aus einem Raspberry Pi 3B+, einer Anzeige vor Ort und einer Klemmenebene mit Relais und Analogein- und -ausgängen. Es läuft ohne Bedienung, ohne Bildschirm und ohne angemeldeten Benutzer, startet nach Stromausfall von selbst und ist über das Netz erreichbar.

Drei Themen prägen diesen Band, und alle drei sind übertragbar auf andere Vorhaben: die Frage, was ein Programm zu einem Dauergerät macht; ein Displaytreiber, der ohne jede Fremdbibliothek auskommt; und die Anbindung einer industriellen Klemmenebene über Modbus RTU.



1. Was ein Programm zu einem Gerät macht

Ein Programm, das man startet, und ein Gerät, das läuft, unterscheiden sich nicht im Algorithmus, sondern in vier Eigenschaften. Sie klingen banal und entscheiden doch darüber, ob die Sache nach vier Wochen noch arbeitet.

Es muss ohne Anmeldung starten. Ein Programm im Autostart eines Benutzers läuft erst, wenn sich jemand anmeldet — auf einem Gerät ohne Bildschirm also nie. Der Wächter läuft deshalb als Systemdienst, gestartet von `systemd`, eingehängt in `multi-user.target`, das lange vor jeder Anmeldung erreicht wird.

Es muss Fehler überleben. Jeder Absturz, jede unbehandelte Ausnahme, jeder Speicherfehler beendet ein gewöhnliches Programm endgültig. Der Dienst trägt `Restart=always` und `RestartSec=15`: Endet der Prozess aus welchem Grund auch immer, wird er nach fünfzehn Sekunden neu gestartet. Das ist keine Entschuldigung für schlechte Fehlerbehandlung, sondern die letzte Rückfallebene.

Es muss mit fehlender Umgebung zurechtkommen. Die Kamera ist solarbetrieben und wochenweise stromlos, das Display kann abgezogen sein, der RS485-Wandler ebenso. Jede dieser Baugruppen ist im Programm so behandelt, dass ihr Fehlen eine Meldung erzeugt und keinen Abbruch. Der Grundsatz: Was Zugabe ist, darf den Kern nicht mitreißen.

Es darf sein Speichermedium nicht auffressen. Bei einem Bild alle fünf Minuten entstehen rund 300 Bilder oder vier Megabyte pro Tag. Ohne Gegenmaßnahme wäre die Speicherkarte in einem Jahr voll — mit dem unangenehmen Fehlerbild, dass das Gerät scheinbar grundlos den Dienst versagt. Eine tägliche Aufräumroutine löscht Bilder, die älter als eine einstellbare Frist sind.

Die Wahl des Unterbaus

Der Pi 3B+ hat 1 GB Arbeitsspeicher. Das ist die eigentliche Randbedingung. Sein Prozessor ist 64-bit-fähig, doch 64-Bit-Code belegt mehr Speicher, weil jede Adresse doppelt so breit ist; bei laufender Oberfläche führt das zu ständigem Auslagern auf die Speicherkarte. Gewählt wurde deshalb die 32-Bit-Ausgabe.

Ebenso begründet ist das Startverhalten: Das Gerät startet in die Textanzeige, nicht in den Desktop. Das spart rund 300 MB und ist zugleich die Voraussetzung dafür, dass der Fern-Desktop über xrdp zuverlässig arbeitet — eine automatisch angemeldete lokale Sitzung belegt sonst die Grafik, und die Verbindung bricht unmittelbar nach dem Aufbau wieder ab. Der Desktop entsteht auf diese Weise genau dann, wenn sich jemand verbindet.

2. Die Anzeige vor Ort

Ein Gerät im Gewächshaus soll seinen Zustand zeigen, ohne dass man einen Browser öffnet. Dafür sitzt ein einfarbiges OLED mit 128×64 Bildpunkten am I2C-Bus.

Warum ein eigener Treiber

Naheliegend wäre eine der verbreiteten Bibliotheken gewesen, luma.oled oder Adafruit-Blinka. Dagegen sprach der Unterbau: Raspberry Pi OS lässt seit Bookworm keine Installation von Python-Paketen ins System mehr zu — der Paketverwalter meldet „externally managed environment“. Bibliotheken müssten in eine virtuelle Umgebung, und der Systemdienst müsste diese Umgebung aktivieren. Für ein Gerät, das jahrelang unbeaufsichtigt laufen soll, ist das eine zusätzliche Bruchstelle.

Die Alternative kostet etwa hundert Zeilen. Das Display braucht ausschließlich Schreibzugriffe, und die beherrscht der Kernaltreiber `i2c-dev` unmittelbar:

```
fd = os.open("/dev/i2c-1", os.O_RDWR)
fcntl.ioctl(fd, 0x0703, adresse)      # I2C_SLAVE
os.write(fd, bytes([0x00]) + befehle)  # 0x00 = es folgen Befehle
os.write(fd, bytes([0x40]) + bilddaten) # 0x40 = es folgen Bilddaten
```

Damit hat die Anzeige keine einzige Zusatzabhängigkeit. Gezeichnet wird mit Pillow, das für die Bildauswertung ohnehin vorhanden ist.

Vom Bild zum Bildspeicher

Der SSD1306 verwaltet seinen Speicher in acht Seiten zu je 128 Byte. Ein Byte beschreibt eine Spalte von acht übereinanderliegenden Bildpunkten, Bit 0 ist der oberste. Aus einem Pillow-Bild wird der Bildspeicher also durch eine Umsortierung:

$$\text{Byte}(s, x) = \sum_{i=0}^7 b(x, 8s + i) \cdot 2^i$$

mit $b \in \{0, 1\}$ als Bildpunkt und s als Seitennummer. In numpy ist das eine Zeile, ohne numpy eine Schleife — beide Wege sind vorhanden und liefern nachweislich dasselbe Ergebnis.

Zwei Controller sind im Umlauf und äußerlich nicht zu unterscheiden. Der SSD1306 beherrscht waagerechte Adressierung, der Bildspeicher lässt sich in einem Zug schreiben. Der SH1106 verwaltet intern 132 Spalten, von denen die mittleren 128 sichtbar sind, kennt diese Adressierung nicht und wird deshalb Seite für Seite mit einem Versatz von zwei Spalten beschrieben. Beide Wege sind eingebaut, umschaltbar über eine Konstante. Das Fehlerbild bei falscher Wahl ist eindeutig: Streifen oder ein Versatz um genau zwei Bildpunkte.

Ein Layout, das in 64 Punkte passt

Auf so kleiner Fläche ist jede Zeile gezählt. Maßgeblich ist die Zeilenhöhe der Schrift, also die Summe aus Ober- und Unterlänge. Bei DejaVu Sans ergibt sich gemessen:

Schriftgröße	Zeilenhöhe
7	9 Punkte
8	10 Punkte
10	13 Punkte
16 fett	19 Punkte

Daraus entstand die Aufteilung der ersten Seite: Kopfzeile 0 bis 9, Trennstrich auf 10, die große Aussage in einem Band von 12 bis 32, darunter drei Zeilen bei 34, 46 und 55. Die letzte endet damit genau auf 64. Der erste Entwurf hatte diese Rechnung nicht angestellt und schnitt die Fußzeile ab — ein Fehler, der auf dem Bildschirm des Entwicklers unsichtbar bleibt, wenn man nicht maßstäblich vorschaut.

Die eigentliche Aussage steht in einer großen Zeile: Wasser ok, bald giessen oder GIESSEN!. Da ein einfarbiges Display keine Ampelfarben kennt, wird der Gießbefehl invertiert dargestellt — heller Balken, dunkle Schrift. Das ist aus einigen Metern erkennbar und ersetzt das Rot.

Drei Seiten wechseln alle sechs Sekunden: Pflanze, Technik, Verlauf. Bei jedem Wechsel verschiebt sich der gesamte Inhalt um einen Bildpunkt. Diese Kleinigkeit beugt dem Einbrennen vor, dem einzigen Verschleiß, den ein OLED im Dauerbetrieb kennt.

3. Die Klemmenebene: Modbus RTU über RS485

Mit dem Waveshare „Modbus RTU Module (A)“ bekommt das Gerät Hände und weitere Sinne: zwei Relais, zwei Digitaleingänge, zwei Analogeingänge und zwei Analogausgänge.

RS485 in drei Sätzen

RS485 überträgt symmetrisch über ein Adernpaar A und B; gewertet wird die Spannungsdifferenz, weshalb Störungen, die beide Adern gleich treffen, herausfallen. Der Bus ist ein Strang, an dem viele Teilnehmer hängen dürfen, aber immer nur einer sendet — es gibt genau einen Master, der fragt, und Teilnehmer, die antworten. Die Masse gehört mitverbunden, auch wenn es ohne sie oft zunächst funktioniert: Ohne gemeinsamen Bezug driften die Pegel, und das Fehlerbild sind sporadische Aussetzer, die man wochenlang der Software anlastet.

Aus der Ein-Master-Regel folgt eine Betriebsvorschrift für diese Anlage: Der Pi und das vorhandene RS485-TO-ETH-Gateway dürfen nicht gleichzeitig arbeiten. Zwei Master erzeugen keine saubere Fehlermeldung, sondern Antworten, die zufällig richtig oder falsch aussehen.

Der Vierfachwandler

Verwendet wird ein Waveshare „USB TO 4CH RS485“ mit dem Baustein WCH CH344. Er meldet sich als ein USB-Gerät (1a86:55d5) und erzeugt vier voneinander unabhängige, galvanisch getrennte Schnittstellen. Unter Linux erscheinen sie als vier Einträge:

```
/dev/serial/by-id/usb-WCH.CN_USB_Quad_Serial_...-if00 = Kanal 1
...-if02      = Kanal 2
...-if04      = Kanal 3
...-if06      = Kanal 4
```

Die Endungen springen in Zweierschritten, weil die ungeraden Nummern zu den Steuer-schnittstellen des Bausteins gehören. Wichtig ist der Weg über /dev/serial/by-id/ und nicht über /dev/ttyUSB0: Die laufende Nummer der tty-Geräte hängt von der Reihenfolge des Einsteckens ab und kann nach einem Neustart eine andere sein. Der Name unter by-id bleibt gleich.

Das Programm probiert alle gefundenen Schnittstellen der Reihe nach durch und verwendet die, an der ein Modul antwortet. Damit ist gleichgültig, an welchem Kanal geklemmt wurde.

Der Telegrammaufbau

Modbus RTU ist von entzweigender Einfachheit. Ein Telegramm besteht aus Adresse, Funktionscode, Nutzdaten und einer Prüfsumme:

[Adresse][Funktion][Daten ...][CRC16 niederwertig][CRC16 höherwertig]

Die Prüfsumme ist ein CRC16 mit dem Polynom 0xA001, rückwärts gerechnet, mit 0xFFFF als Startwert — dieselbe Routine, die im Wächter schon für die Bildblöcke der Kamera dient:

```
def crc16(daten):
    crc = 0xFFFF
    for b in daten:
        crc ^= b
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc
```

Verwendet werden sechs Funktionscodes:

Code	Bedeutung	hier benutzt für
01	Coils lesen	Zustand der Relais
02	Diskrete Eingänge lesen	Digitaleingänge
03	Haltereister lesen	Analogausgänge, Messbereich, Version
04	Eingangsregister lesen	Analogeingänge
05	Einzelne Coil schreiben	Relais schalten
06	Einzelnes Register schreiben	Analogausgang, Messbereich

Ein Beispiel: Relais 1 einschalten heißt Adresse 1, Funktion 5, Coil 0x0000, Wert 0xFF00. Daraus wird die Bytefolge 01 05 00 00 FF 00 plus zwei Byte Prüfsumme. Der Wert 0xFF00 für „ein“ und 0x0000 für „aus“ ist eine Eigenheit von Modbus, die sich aus der Frühzeit des Protokolls erklärt und keine tiefere Bedeutung hat.

Registerplan des Moduls

Größe	Adresse	Zugriff	Einheit
Relais 1 / 2	Coil 0x0000 / 0x0001	FC01 lesen, FC05 schreiben	0xFF00 = ein
Digitaleingang 1 / 2	0x0000, 2 Bit	FC02	0 / 1
Analogeingang 1 / 2	0x0000, 2 Register	FC04	mV oder µA, je Messbereich
Analogausgang 1 / 2	0x0000, 2 Register	FC03 / FC06	µA, 0 bis 20000
Messbereich AI 1 / 2	0x3000 / 0x3001	FC03 / FC06	siehe unten
Firmware-Version	0x8000	FC03	Wert / 100

Die Analogausgänge können ausschließlich Strom, 0 bis 20 mA, angegeben in Mikroampere. Eine Spannungsausgabe gibt es nicht; wo sie gebraucht wird, erzeugt man sie mit einem Bürdewiderstand.

Die Analogeingänge dagegen sind umschaltbar:

Wert	Messbereich	Einheit des Rohwerts
0	0-5 / 0-10 V	mV
1	1-5 / 2-10 V	mV
2	0-20 mA	µA
3	4-20 mA	µA
4	ADC roh	—

Zwei Dinge gehören zusammen: das Register und ein Schiebeschalter auf der Platine. Für Spannungsmessung muss der Schalter des Kanals auf OFF stehen, sonst liegt der Bürdewiderstand parallel zum Eingang und verfälscht die Messung. Das Programm schreibt den gewünschten Bereich beim Verbinden selbst ins Modul, liest ihn zur Kontrolle zurück und rechnet die Rohwerte entsprechend um — mV zu Volt, µA zu Milliampere. In der Anzeige steht der Bereich in Klammern hinter dem Messwert, damit kein Zweifel entsteht, was die Zahl bedeutet.

Abfragefaden und Sperre

Die Anzeige soll in Echtzeit mitlaufen, während gleichzeitig aus dem Browser geschaltet werden kann. Beides greift auf dieselbe serielle Leitung zu, und Modbus RTU verträgt keine Überlappung: Zwischen Frage und Antwort darf nichts dazwischenfunken.

Gelöst ist das mit einem eigenen Abfragefaden, der im Sekundentakt alle vier Größen liest, und einer Sperre, die jeden Zugriff umschließt — auch die Schaltbefehle aus dem Webserver. Die Wartezeit, die ein Bedienbefehl dadurch erfährt, liegt im Bereich einiger Millisekunden und ist nicht wahrnehmbar.

Fällt die Verbindung aus, wird die Schnittstelle geschlossen, im Zustand vermerkt und alle fünf Sekunden neu gesucht. Das Abziehen des Wandlers im laufenden Betrieb ist damit kein Störfall, sondern eine Meldung im Dashboard.

Die Gießfunktion und ihre Sicherungen

Relais 2 schaltet die Pumpe oder das Ventil. Eine Pumpe, die eingeschaltet bleibt, ist der einzige Fehler in diesem ganzen Programm, der wirklich Schaden anrichten kann — entsprechend ist die Funktion abgesichert.

Die Dauer ist auf zehn Minuten begrenzt, unabhängig davon, was übergeben wird. Ein zweiter Aufruf während eines laufenden Vorgangs wird abgewiesen, statt die Zeit zu verlängern. Das Ausschalten steht in einem `finally`-Block, wird also auch dann ausgeführt, wenn dazwischen etwas fehlschlägt, und wird bis zu fünfmal wiederholt, falls die Leitung gerade stört. Und der Vorgang läuft in einem eigenen Faden, damit der Webserver währenddessen bedienbar bleibt.

Bewusst nicht eingebaut ist die selbsttätige Auslösung durch die Ampel. Wann ein Gerät ohne Aufsicht Wasser laufen lässt, unter welchen Bedingungen und mit welcher Sperrzeit danach, ist eine Entscheidung, die der Betreiber treffen muss und die von der Bewässerungstechnik abhängt. Die Mechanik dafür steht bereit; die Regel fehlt noch.

4. Das erweiterte Dashboard

Die Webseite hat eine zusätzliche Karte bekommen, die den Zustand der Klemmenebene zeigt und bedienbar macht. Der Aufbau folgt dem, was schon vorhanden war: Die Seite fragt im Dreisekundentakt /daten ab und bekommt ein JSON-Paket, das nun einen Abschnitt io enthält.

Neue Endpunkte:

Aufruf	Wirkung
/relais?nr=1&ein=1	Relais schalten
/ao?kanal=1&wert=12000	Analogausgang setzen, Wert in μA
/giessen?sekunden=20	Relais 2 zeitbegrenzt einschalten

Zwei Feinheiten der Bedienung verdienen Erwähnung. Der Zustand der Relais wird nicht mitgeschrieben, sondern aus dem Modul zurückgelesen — die Anzeige zeigt also, was tatsächlich geschaltet ist, nicht was befohlen wurde. Und die Eingabefelder der Analogausgänge werden zwar laufend mit dem gemessenen Wert nachgeführt, aber nicht, während der Mauszeiger darin steht; sonst würde die eigene Eingabe im Sekundentakt überschrieben.

5. Erfahrungen aus der Inbetriebnahme

Die Einrichtung kostete deutlich mehr Zeit als geplant, und zwar nicht wegen des Programms. Drei Punkte sind festgehalten, weil sie sich jederzeit wiederholen können und von außen nicht zu erkennen sind.

Der Raspberry Pi Imager in der Fassung 2.0.10 schrieb die Voreinstellungen — Benutzer, WLAN, Fernzugriff — nicht auf die Karte, meldete aber „Schreiben erfolgreich“. Der Pi startete daraufhin ohne Benutzer und ohne Netz. Von außen ist das nicht von einem Verdrahtungsfehler oder einem vertippten Kennwort zu unterscheiden. Der Fehler ist für mehrere Fassungen der 2.0-Reihe in den Fehlerberichten des Projekts belegt.

Dieselbe Fassung stellte zudem die Beschriftungen der Schaltflächen verstümmelt dar: Die eingebettete Schrift wurde mit einer um zwei Plätze verschobenen Zeichentabelle geladen, aus SPEICHERN wurde QNCGAF CPL. Auch das ist belegt und in der Nachfolgefassung behoben.

Und schließlich läuft die Ersteinrichtung eines Raspberry Pi genau ein Mal. Ausgelöst wird sie durch einen Eintrag in cmdline.txt, den das Einrichtungsprogramm anschließend selbst entfernt. Wer eine Konfigurationsdatei erst nach dem ersten Einschalten nachträgt, wartet vergeblich — sie wird nie gelesen. Die Reihenfolge ist zwingend: brennen, Datei kopieren, dann erst zum ersten Mal einschalten.

Aus allen dreien folgt dieselbe Lehre, die auch in anderen Projekten trägt: Sobald eine Diagnose von außen nicht mehr möglich ist, lohnt der Aufwand, sich Sicht zu verschaffen — hier ein Monitor und eine Tastatur für die Ersteinrichtung. Die Alternative ist das Durchprobieren von Vermutungen, und das kostet mehr Zeit, als das Kabel je gekostet hätte.

6. Dateien

Alles, was für einen vollständigen Neuaufbau gebraucht wird. Diese Dateien gehören außerhalb des Geräts aufbewahrt — auf dem PC und zusätzlich auf einem Datenträger, den kein Kartenschaden erreicht.

Auf dem Raspberry Pi, Ordner /home/pi/tomate

Datei	Inhalt
pflanzen_dashboard.py	Hauptprogramm: Bildabholung, Auswertung, Webserver, Anbindung von Display und Klemmenebene
oled_anzeige.py	Displaytreiber SSD1306/SH1106 und Seitengestaltung
waveshare_io.py	Modbus-RTU-Anbindung des Moduls A, Abfragefaden, Gießfunktion
pflanzen_waechter.service	Vorlage des Systemdienstes
install_pi.sh	Einrichtungsskript: Pakete, I2C, Autostart

Werkzeuge zur Inbetriebnahme

Datei	Zweck
waveshare_wandler_test.py	prüft den RS485-Wandler allein, mit Schleifentest zwischen zwei Kanälen
waveshare_pi_test.py	prüft das Modul A: Version, DI, AI, AO, Relais

Auf der Kamera

Datei	Inhalt
ESP32_CAM_Modbus_Snapshot.ino	Sketch der ESP32-CAM mit Blockübertragung, Statusregistern und OTA

Dokumentation

Datei	Inhalt
MANUSKRIPT_Tomatenwaechter.md	Band 1: Verfahren, Protokoll, Bildauswertung, Bedienung
MANUSKRIPT_Tomatenwaechter_Pi.md	Band 2: dieses Dokument
ANLEITUNG_Pi_OLED.md	Arbeitsanleitung: Installation, Betrieb, Fehlersuche, Wiederaufbau

Betriebsdaten, die das Gerät selbst anlegt

pflanzen_log.csv mit der Messreihe, pflanzen_referenz.json mit der gesetzten Referenz und der Ordner pflanzen_bilder. Die Messreihe ist der einzige unwiederbringliche Teil und sollte gelegentlich auf den PC geholt werden.

7. Ausblick

Drei Erweiterungen liegen nahe und sind vorbereitet.

Die selbsttätige Bewässerung braucht nur noch eine Regel: bei welcher Ampelfarbe, für wie lange, mit welcher Sperrzeit danach, und ob nur bei Tageslicht. Sinnvoll ist eine Sperrzeit von mehreren Stunden, denn eine Pflanze braucht Zeit, um auf Wasser zu reagieren — ohne diese

Sperre würde das Gerät nachgießen, bevor die erste Gabe wirken konnte, und der Regelkreis schwingt.

Ein Bodenfeuchtesensor am Analogeingang gäbe der Entscheidung ein zweites, unabhängiges Kriterium. Die Blattstellung ist ein später Anzeiger — wenn die Blätter hängen, ist der Wassermangel bereits eingetreten. Die Bodenfeuchte zeigt ihn früher. Beide zusammen erlauben eine Aussage, die keiner von beiden allein liefert: hängende Blätter bei feuchtem Boden deuten nicht auf Durst, sondern auf Wurzelschaden oder Hitze.

Und schließlich ließe sich die zweite Ampel auf dem Display darstellen, sobald zwei Kriterien vorliegen. Platz dafür ist auf der Technikseite.

Anhang: Befehle im Betrieb

Zweck	Befehl
Zustand des Dienstes	<code>systemctl status pflanzen_waechter</code>
Meldungen mitlesen	<code>journalctl -u pflanzen_waechter -f</code>
Nach Programmänderung	<code>sudo systemctl restart pflanzen_waechter</code>
Modul einzeln prüfen	Dienst anhalten, dann <code>python3 ~/tomate/waveshare_pi_test.py</code>
Display einzeln prüfen	Dienst anhalten, dann <code>python3 ~/tomate/oled_anzeige.py demo</code>
I2C-Teilnehmer suchen	<code>i2cdetect -y 1</code>
Serielle Kanäle auflisten	<code>ls /dev/serial/by-id/</code>
Sauber ausschalten	<code>sudo poweroff</code>

Der Dienst belegt die serielle Schnittstelle dauerhaft. Die beiden Prüfprogramme lassen sich deshalb nur bei angehaltenem Dienst verwenden.